UNIVERSIDADE FEDERAL DE SANTA CATARINA
CAMPUS ARARANGUÁ
LIGA PROTOFPGA

Curso Introdutório de Programação de Baixo Nível

PROJETO FINAL

Orientações, requisitos e temas

Abertura e escolha do tema: 6 nov. 2026 · Entrega no GitHub: 19 nov. 2026, até 23h59

1 APRESENTAÇÃO

Em duplas, cada uma com um tema diferente, os participantes escrevem um programa em assembly MIPS que roda no simulador MARS, explicam o programa num arquivo `README.md` e publicam tudo num repositório no GitHub.

O projeto vale horas: as horas do curso, para quem participou dos encontros, mais horas pelo desenvolvimento do repositório. Os requisitos da seção 4 são o mínimo; quanto mais bem feito o repositório (código, comentários, `README.md` e extras), mais horas.

Os temas têm a mesma dificuldade, média, e podem ser feitos no prazo com o que foi visto até o encontro 5: registradores, memória, controle de fluxo e funções.

2 CALENDÁRIO

São menos de duas semanas. Por isso, o tema é escolhido no mesmo dia da abertura, e o trabalho deve começar pela parte mais simples: o teste obrigatório funcionando, com valores fixos no código.

Quadro 1 – Calendário do projeto

Quando	**O quê**
6 nov. 2026	Abertura do projeto e escolha do tema
19 nov. 2026, até 23h59	Entrega do link do repositório no GitHub, com o programa (`.asm`) e o `README.md`; depois disso, avaliação

Fonte: elaborado pelos autores (2026).

3 ESCOLHA DO TEMA

Cada dupla fica com um tema diferente do Quadro 2, e a divisão é feita hoje, na hora. Assim, o curso termina com dez programas diferentes, e não dez versões do mesmo.

4 REQUISITOS OBRIGATÓRIOS

Todo programa, de qualquer tema, precisa cumprir os oito requisitos abaixo. Convém conferir a lista inteira antes de entregar. O programa:

a) monta e roda no MARS sem erro e termina com o serviço 10 (`li $v0, 10` seguido de `syscall`);

b) passa no **teste obrigatório** do tema (Quadro 2) e mostra o resultado no painel Run I/O;

c) exibe o resultado com texto que explique o número, como `Soma = 14`, e não apenas `14`;

d) tem pelo menos **um laço** (encontro 4);

e) tem pelo menos **uma função escrita pela dupla**, chamada com `jal`, que recebe os argumentos em `$a0`–`$a3`, devolve o resultado em `$v0` (quando houver) e volta com `jr $ra` (encontro 5);

f) respeita a convenção de chamada: o que precisa sobreviver a um `jal` fica num registrador `$s`, e uma função que chama outra guarda o `$ra` na pilha;

g) começa com um cabeçalho de comentários (nomes da dupla, tema e o que o programa faz) e tem comentários nas partes principais, como os arquivos das aulas;

h) está num repositório público no GitHub e vem acompanhado de um `README.md` no modelo da seção 6.

Podem ser usadas todas as instruções e chamadas de sistema do MARS, inclusive as pseudoinstruções (`li`, `la`, `move`…), e podem ser consultados as apostilas, o Apêndice A e os programas das aulas. O código, porém, precisa ser da dupla, e os dois devem saber explicar qualquer trecho dele.

5 TEMAS

O Quadro 2 traz os dez temas. A função sugerida é apenas uma ideia: a dupla pode escrever outra, desde que o programa tenha pelo menos uma. A coluna “Para ir além” é opcional, mas um repositório mais completo vale mais horas.

Quadro 2 – Temas do projeto final

Nº	Tema	O que o programa faz	Teste obrigatório	Para ir além
1	Calculadora acumuladora (A.6)	Lê inteiros até receber 0 e imprime a soma, quantos números foram lidos, o maior e o menor. Função sugerida: `imprime(texto, valor)`.	5, 12, −3, 0 → `Soma = 14`, `Quantidade = 3`, `Maior = 12`, `Menor = -3`	Mostrar também a média inteira
2	Fibonacci	Lê N e imprime os N primeiros termos (cada termo é a soma dos dois anteriores). Função sugerida: `fibonacci(n)`, que imprime os termos.	10 → `0 1 1 2 3 5 8 13 21 34`	Versão recursiva, `fib(n)`

Nº	Tema	O que o programa faz	Teste obrigatório	Para ir além
3	Sequência de Collatz	Lê N. Enquanto N não for 1: se for par, divide por 2; se for ímpar, faz 3N + 1. Imprime a sequência e o número de passos. Função sugerida: `proximo(n)`.	6 → `6 3 10 5 16 8 4 2 1` e `Passos = 8`; 7 → 16 passos	Qual N de 1 a 30 tem a sequência mais longa
4	MDC e MMC	Lê dois inteiros positivos e imprime o MDC (Euclides: troca a e b por b e o resto de a ÷ b, até b virar 0) e o MMC = a × b ÷ MDC. Função sugerida: `mdc(a, b)`.	12 e 18 → `MDC = 6` e `MMC = 36`	Versão recursiva de `mdc`
5	Números primos (A.9)	Lê N e imprime os N primeiros primos. Função obrigatória: `eh_primo(n)`, que devolve 1 ou 0.	10 → `2 3 5 7 11 13 17 19 23 29`	Os cem primeiros, dez por linha, como pede o A.9
6	Decimal para binário	Lê um inteiro positivo e o imprime em binário: guarda os restos da divisão por 2 num vetor e os imprime de trás para a frente. Função sugerida: `converte(n, vetor)`, que devolve quantos dígitos gerou.	13 → `1101`; 255 → `11111111`	Converter para qualquer base de 2 a 9
7	Dígitos por extenso (A.8)	Lê um inteiro positivo e imprime cada dígito por extenso. Dica: um vetor de endereços de textos, `.word zero, um, dois`…. Função sugerida: `imprime_digito(d)`.	2026 → `dois zero dois seis`	Aceitar negativos: −7 → `menos sete`
8	Número palíndromo	Lê um inteiro positivo, inverte os dígitos e diz se o número se lê igual nos dois sentidos. Função sugerida: `inverte(n)`.	12321 → `12321 e palindromo`; 1234 → `1234 nao e palindromo (invertido: 4321)`	Listar os palíndromos de 100 a 200
9	Estatísticas de notas	Lê dez notas inteiras (0 a 10) para um vetor e imprime soma, média inteira, maior, menor e quantos aprovados (nota ≥ 6). Função sugerida: uma por estatística, como `maior(vetor, n)`.	7 4 9 6 3 10 5 8 6 2 → soma 60, média 6, maior 10, menor 2, aprovados 6	Imprimir as notas acima da média
10	Jogo de adivinhação	Sorteia um número de 1 a 100; a pessoa chuta até acertar; a cada chute, o programa responde `maior` ou `menor`; no fim, mostra quantas tentativas foram. Função sugerida: `compara(chute, segredo)`, que devolve −1, 0 ou 1.	Com o segredo fixo em 37, só para testar: chutes 50, 25, 37 → `menor`, `maior`, `Acertou em 3 tentativas`	Limitar a 7 tentativas e revelar o número se a pessoa perder

Fonte: elaborado pelos autores (2026).

Os textos do programa devem ser escritos sem acento (`nao e palindromo`, e não `não é palíndromo`), porque o MARS pode exibir letras acentuadas de forma estranha no painel Run I/O.

6 O QUE ENTREGAR

Cada dupla cria um **repositório público no GitHub** com os dois arquivos do Quadro 3. A entrega é o envio do link do repositório ao ministrante até 19 nov. 2026, às 23h59; até lá, o repositório precisa conter a versão final. Depois da avaliação, os links serão reunidos no repositório da liga.

Quadro 3 – Itens da entrega

Item	Nome	Conteúdo
Repositório	Público no GitHub, por exemplo `projeto-assembly-collatz`	Os dois arquivos abaixo, na raiz
Programa	`projeto_<tema>.asm`, por exemplo `projeto_collatz.asm`	O código, com o cabeçalho de comentários
Documentação	`README.md`	O modelo do Código 1, preenchido; o GitHub o exibe na página inicial do repositório

Fonte: elaborado pelos autores (2026).

O `README.md` é o que alguém lê antes de abrir o código. Basta copiar o modelo do Código 1 e trocar o que está entre `< >`.

Código 1 – Modelo de README.md

```
# <Nome do tema>

Projeto final do Curso Introdutorio de Programacao de Baixo Nivel
Liga ProtoFPGA - UFSC Ararangua

**Dupla:** <nome 1> e <nome 2>

## O que o programa faz
<Duas ou tres frases, em linguagem simples.>

## Como executar
1. Abra `projeto_<tema>.asm` no MARS.
2. Clique em Assemble e depois em Run.
3. <O que digitar no painel Run I/O, se o programa pedir algo.>

## Exemplo
Entrada: <o que foi digitado>
Saida: <o que apareceu no Run I/O>

## Como o codigo esta organizado
| Funcao   | Recebe        | Devolve       | O que faz |
| -------- | ------------- | ------------- | --------- |
| `<nome>` | `$a0` = <...> | `$v0` = <...> | <...>     |

## Limitacoes
<O que o programa nao trata: negativos, valores muito grandes...>
```

Fonte: elaborado pelos autores (2026).

6.1 Como publicar no GitHub

Quem nunca usou git pode fazer tudo pelo site, sem linha de comando:

a) criem uma conta em github.com; o repositório fica na conta de um dos dois integrantes;

b) cliquem em **New repository**, deem o nome (por exemplo `projeto-assembly-collatz`), marquem **Public** e cliquem em **Create repository**;

c) na página do repositório, cliquem em **uploading an existing file** (ou **Add file → Upload files**), arrastem o `.asm` e o `README.md` e cliquem em **Commit changes**;

d) em **Settings → Collaborators**, adicionem o colega de dupla, para que os dois possam enviar mudanças;

e) para corrigir um arquivo, enviem-no de novo do mesmo jeito: o GitHub guarda cada versão.

Quem já usa git pode trabalhar pelo terminal (`git clone`, `git add`, `git commit`, `git push`). Nos dois casos, convém fazer um commit a cada parte que funcionar: se algo quebrar, é possível voltar à versão anterior.

Dificuldade com git ou GitHub? A liga ajuda. Basta falar com o ministrante antes de o prazo apertar: ninguém deve deixar de entregar por causa disso.